1.CPSC 304 Project Cover Page

Milestone #: 2

Date: Oct 25, 2021

Group Number: 91

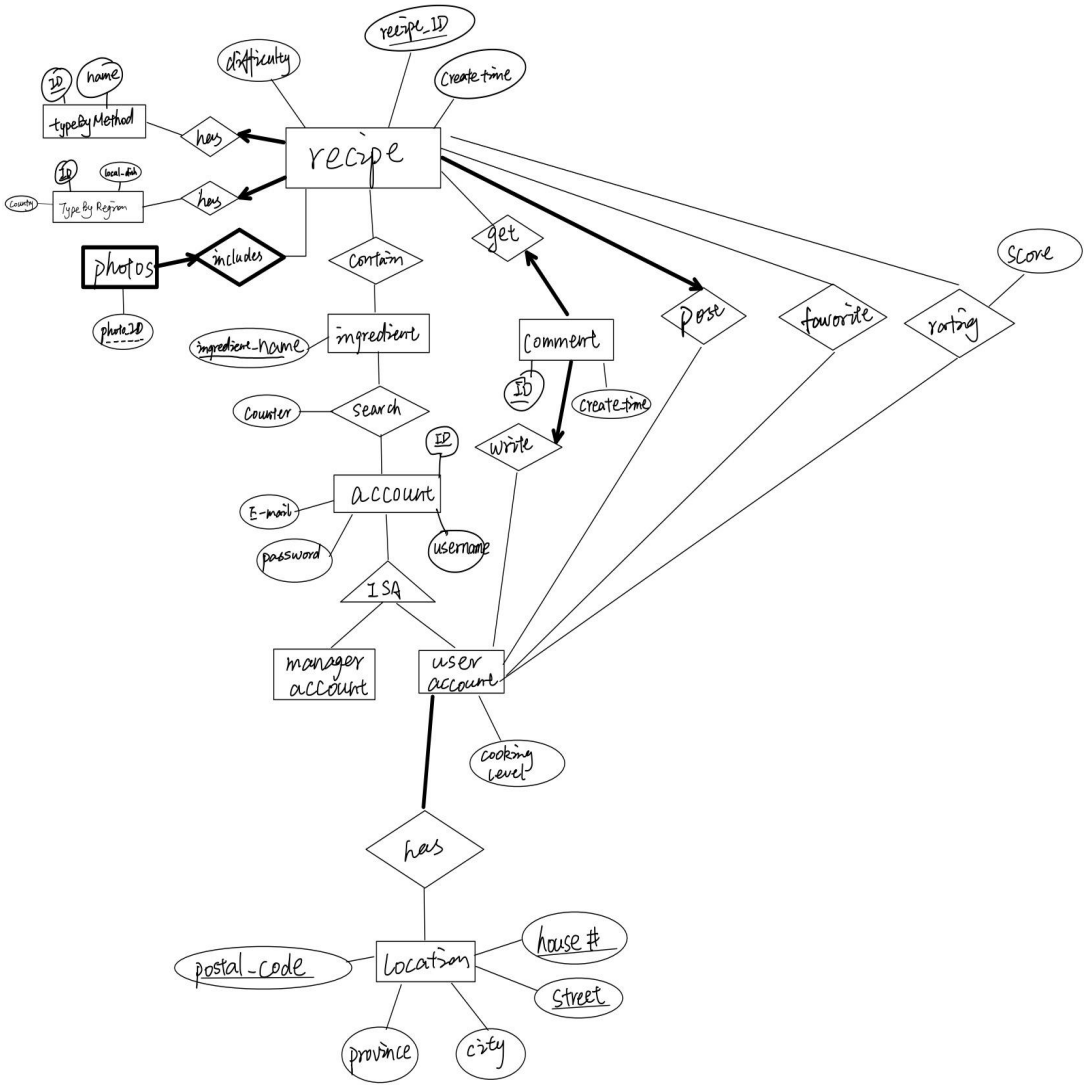
Name	Student Number	CS Alias (Userid)	Preferred Email Address
Yining Cheng	61580544	a8g3b	carol7102@hotmail.com
Qin(Jean) Xue	72070014	u6e2b	jeanxue0716@gmail.com
Miao Zhang	68167618	v1z2b	tresaguas1317@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above. (In the case of Project Milestone 0, the main purpose of this page is for you to let us know your email address, and then let us assign you to a TA for your project supervisor.)

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

2. ER Diagram & Descriptions on Changes

M1 recipe



There are seven changes in the diagram:

1. We delete the “user_id” for the **recipe** entity because it is a foreign key from the **userAccount** entity in the post relationship.
2. We change the name of the attribute of the **ingredient** entity to “ingredient_name” to show the name of the ingredient.
3. We change the **comment** relationship to the comment entity and let “ID” be its key. There are many-to-1 relationships: one recipe has many comments and one user account can post many comments, also all comments are in recipes and posted by users. We change here because if a comment is a relationship, one

user can comment on a recipe at most once. In the new ER diagram, the problem is solved, any user can comment on a recipe many times.

4. We change the **rating** entity to **rating** relationship since if **rating** is an entity, the users can rate a recipe many times, it does not make sense in the real world. We solve it by making it a many-to-many relationship to guarantee there is only one rating for the exact recipe rated by the exact user.
5. We change the **countryType** by adding a non-key attribute “local_dish” and change its name to typeByRegion. We add the attribute “local_dish”. The attribute “local_dish” can help users to find their wanted cuisine more precisely.
6. We made the date as an entity for allowing users to search the same ingredient many times.
7. We add some additional attributes on **location:** postal code, house number, street. These additives are there for collecting more specific user information. This extra information can provide some functions that satisfy users’ demand.

3&4. Schema & Functional Dependencies

T1
recipe(ID: integer, difficulty: integer, createtime: integer, user_ID: integer, typeByMethod_ID: integer, typeByRegion_ID: integer)
Primary key: recipe_ID
Candidate key: recipe_ID
Foreign key: (user_ID, typeByMethod_ID, typeByRegion_ID)
Constraints: user_ID, typeByMethod_ID, typeByRegion_ID are not null
FDs: ID -> (difficulty, createtime, user_ID, typeByMethod_ID, typeByRegion_ID)

T2
typeByMethod(ID: integer, method: string)
Primary key: ID
Candidate key: ID
Foreign key:
Constraints:
FDs: ID -> method

T3
typeByRegion(ID: integer, local_dish: string, country: string)
Primary key: ID
Candidate key: ID
Foreign key:
Constraints:
FDs: ID -> local_dish
Local_dish -> country

T4
photos_includes(ID: integer, recipe_ID)
Primary key: (ID, recipe_ID)
Candidate key: (ID, recipe_ID)
Foreign key: recipe_ID
Constraints:
FDs:

T5
rating(recipe_ID: integer, user_ID: integer, score: integer)
Primary key: (recipe_ID, user_ID)
Candidate key: (recipe_ID, user_ID)
Foreign key: (recipe_ID, user_ID)
Constraints:
FDs: (recipe_ID, user_ID) -> score

T6
ingredient(ingredient_name: char(20))
Primary key: (ingredient_name)
Candidate key: (ingredient_name)
Foreign key: NaN
Constraints: NaN
FDs: ingredient_name -> ingredient_name

T7
contain(ingredient_name: char(20), recipe_ID:integer)
Primary key: (ingredient_name,recipe_ID)
Candidate key: (ingredient_name,recipe_ID)
Foreign key: ingredient_name,recipe_ID
Constraints:
FDs: ingredient_name,recipe_ID -> ingredient_name,recipe_ID

T8
account(account_ID:integer, username: char(20), Email:char(30), password:char(30))
Primary key: account_ID
Candidate key: (Email),(account_ID)
Foreign key:
Constraints: Email is unique
FDs: account_ID->Email, password, username
Email-> account_ID, password, username

T9
search(account_ID:integer, counter: integer, ingredient_name:char(20))
Primary key: (account_ID, ingredient-name)

Candidate key: (account_ID, ingredient-name)
Foreign key: account_ID, ingredient-name
Constraints:
FDs: account_ID, ingredient-name -> account_ID, ingredient-name

T10
comment(id: integer, createTime: integer, recipe_ID: integer, user_account_id: integer)
Primary key: id
Candidate key: id
Foreign key: recipe_id, user_account_id
Constraints: user_account_id is not NULL and recipe_ID is not NULL
FDs: id->create_time, recipe_ID, user_account_id

T11
favorite(user_account_id: integer, recipe_id: integer)
Primary key: (user_account_id, recipe_id)
Candidate key: (user_account_id, recipe_id)
Foreign key: (user_account_id, recipe_id)
Constraints:
FDs:

T12
location(postal_code: string, house#: integer, street: string, province: string, city: string)
Primary key: (postal_code, house#, street)
Candidate key: (postal_code, house#, street)
Foreign key:
Constraints:
FDs:postal_code->province, city
(postal_code, house#, street)->province, city

T13
user_account(user_account_id: integer, cooking_level: integer, postal_code: string, house#: integer, street: string)
Primary key: (user_account_id)
Candidate key: user_account_id
Foreign key: postal_code, house#, street, user_account_id
Constraints: house# cannot be NULL
postal_code cannot be NULL
Street cannot be NULL

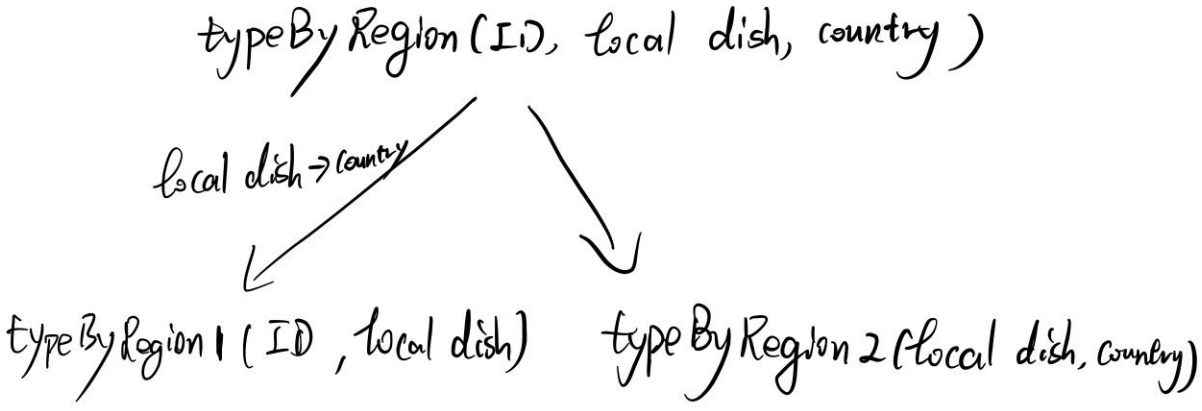
FDs: user_account_id->cooking_level, postal_code, house#, street

T14
managerAccount: (manager_account_id: integer)

Primary key: manager_account_id
Candidate key: manager_account_id
Foreign key:manager_account_id
Constraints:
FDs:

5. Normalization

a) For table3 **typeByRegion**(ID: integer, local_dish: string, country: string)
FDs: ID -> local_dish
local_dish -> country
The closure of local_dish is country, not includes all attributes, so local_dish is not superkey and country is a non-key attribute.
⇒the FD violates the 3NF and BCNF
We use BCNF to normalize it,



b)

For table location(

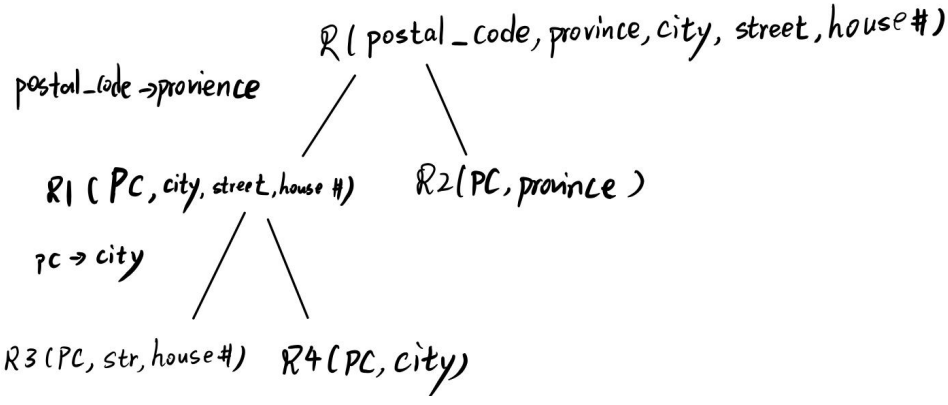
FDs: PC → province, city

PC = postal code

{PC, street, House #}$^+$ = R.

The closure of PC does not includes all attributes, so PC is not superkey and city and province are non-key attribute.

=> FD violates the 3NF and BCNF, we use BCNF to normalize it.



6. SQL DDL(with table after normalization)

T1

recipe(ID: integer, difficulty: integer, createtime: integer, user_ID: integer, typeByMethod_ID: integer, typeByRegion_ID: integer)

Primary key: recipe_ID

Candidate key: recipe_ID

Foreign key: (user_ID, typeByMethod_ID, typeByRegion_ID)

Constraints: user_ID, typeByMethod_ID, typeByRegion_ID are not null

FDs: ID -> (difficulty, createtime_ID, user_ID, typeByMethod_ID, typeByRegion_ID)

```
CREATE TABLE recipe (
        ID              INTEGER PRIMARY KEY,
        DIFFICULTY      INTEGER,
        CREATETIME      INTEGER,
        USER_ID         INTEGER NOT NULL,
        METHOD_ID       INTEGER NOT NULL,
        REGION_ID       INTEGER NOT NULL,
```

```
        FOREIGN KEY (USER_ID) REFERENCES userAccount(ACCOUNT_ID),
        FOREIGN KEY (METHOD_ID) REFERENCES typeByMethod(ID),
        FOREIGN KEY (REGION_ID) REFERENCES typeByRegiona1(ID)

)
```

T2
typeByMethod(ID: integer, method: string)
Primary key: ID
Candidate key: ID
Foreign key:
Constraints:
FDs: ID -> method

```
CREATE TABLE typeByMethod (
        ID INTEGER,
        METHOD CHAR(20),
        PRIMARY KEY (ID)
)
```

T3
typeByRegion1(ID: integer, local_dish: string)
Primary key: ID
Candidate key: ID
Foreign key: local_dish
Constraints:
FDs: ID -> local_dish

```
CREATE TABLE typeByRegion1 (
        ID INTEGER,
        LOCAL_DISH CHAR(20),
        PRIMARY KEY (ID),
        FOREIGN KEY (LOCAL_DISH) REFERENCES typeByRegion2
)
```

T4
typeByRegion2(local_dish: string, country: string)
Primary key: local_dish
Candidate key: local_dish
Foreign key:
Constraints:
FDs: local_dish -> country

```
CREATE TABLE typeByRegion2 (
        LOCAL_DISH CHAR(20),
        COUNTRY CHAR(20),
        PRIMARY KEY (LOCAL_DISH)
)
```

T5
photos_includes(ID: integer, recipe_ID)
Primary key: (ID, recipe_ID)
Candidate key: (ID, recipe_ID)
Foreign key: recipe_ID
Constraints:
FDs:

```
CREATE TABLE photos_includes (
        PHOTO_ID INTEGER,
        RECIPE_ID INTEGER,
        PRIMARY KEY (PHOTOS_ID, RECIPE_ID),
        FOREIGN KEY (RECIPE_ID) REFERENCES recipe,
                ON DELETE CASCADE
                ON UPDATE CASCADE
)
```

T6
rating(recipe_ID: integer, user_ID: integer, score: integer)
Primary key: (recipe_ID, user_ID)
Candidate key: (recipe_ID, user_ID)
Foreign key: (recipe_ID, user_ID)
Constraints:
FDs: (recipe_ID, user_ID) -> score

```
CREATE TABLE rating (
        USER_ID         INTEGER,
        RECEPE_ID       INTEGER,
        SCORE           INTEGER,

        PRIMARY KEY (USER_ID, RECIPE_ID),
        FOREIGN KEY (USER_ID) REFERENCES userAccount(ACCOUNT_ID),
        FOREIGN KEY (RECIPE_ID) REFERENCES recipe(ID)
)
```

T7
ingredient(ingredient_name: char(20))

Primary key: (ingredient_name)
Candidate key: (ingredient_name)
Foreign key: NaN
Constraints: NaN
FDs:
CREATE TABLE ingredient (
INGREDIENT_NAME CHAR(20),

PRIMARY KEY (INGREDIENT_NAME)
)

T8
contain(ingredient_name: char(20), recipe_ID:integer)
Primary key: (ingredient_name,recipe_ID)
Candidate key: (ingredient_name,recipe_ID)
Foreign key: ingredient_name,recipe_ID
Constraints:
FDs:

CREATE TABLE contain (
INGREDIENT_NAME CHAR(20),
RECEPE_ID INTEGER,

PRIMARY KEY (INGREDIENT_NAME , RECIPE_ID),
FOREIGN KEY (INGREDIENT_NAME) REFERENCES
ingredient(INGREDIENT_NAME),
FOREIGN KEY (RECIPE_ID) REFERENCES recipe(ID)
)

T9
account(account_ID:integer, username: char(20), Email:char(30), password:char(30))
Primary key: account_ID
Candidate key: (Email),(account_ID)
Foreign key:
Constraints: Email is unique
FDs: account_ID->Email, password, username
Email-> account_ID, password, username

CREATE TABLE account (
ACOUNT_ID INTEGER,
USERNAME CHAR(20),
E-MAIL CHAR(30),
PASSWORD CHAR(30),

```
        PRIMARY KEY (ACOUNT_ID)
)
```

T10

search(account_ID:integer, counter: integer, ingredient-name:char(20))
Primary key: account_ID, ingredient-name
Candidate key: (account_ID, ingredient-name)
Foreign key: account_ID, ingredient-name
Constraints:
FDs:

```
CREATE TABLE search (
        ACCOUNT_ID              INTEGER
        COUNTER                 INTEGER,
        INGREDIENT_NAME      CHAR(20),

        PRIMARY KEY (ACOUNT_ID, INGREDIENT_NAME),
        FOREIGN KEY (ACCOUNT_ID ) REFERENCES userAccount(ACCOUNT_ID),
        FOREIGN KEY (INGREDIENT_NAME) REFERENCES
                        ingredient(INGREDIENT_NAME)
)
```

T11

comment(id: integer, createTime: integer, recipe_ID: integer, user_account_id: integer)
Primary key: id
Candidate key: id
Foreign key: recipe_id, user_account_id
Constraints: user_account_id is not NULL and recipe_ID is not NULL
FDs: id->(create_time, recipe_ID, user_account_id)

```
CREATE TABLE comment(
        ID                              INTEGER
        CREATE_TIME                     CHAR(30)
        RECIPE_ID                       INTEGER
        USER_ACCOUNT_ID                 INTEGER

        PRIMARY KEY USER_ACCOUNT_ID
        FOREIGN KEY (USER_ACCOUNT_ID ) REFERENCES userAccount(ACCOUNT_ID),
        FOREIGN KEY (RECIPE_ID) REFERENCES,
                        recipe(ID)
)
```

T12
favorite(user_account_id: integer, recipe_id: integer)
Primary key: (user_account_id, recipe_id)
Candidate key: (user_account_id, recipe_id)
Foreign key: (user_account_id, recipe_id)
Constraints:
FDs:

```
CREATE TABLE favorite(
        USER_ACCOUNT_ID          INTEGER,
        RECIPE_ID                INTEGER,

        PRIMARY KEY (USER_ACCOUNT_ID, RECIPE_ID),
        FOREIGN KEY (USER_ACCOUNT_ID ) REFERENCES userAccount(ACCOUNT_ID),
        FOREIGN KEY (RECIPE_ID) REFERENCES recipe(ID)
)
```

T13
location1(postal_code: string, house#: integer, street: string)
Primary key: (postal_code, house#, street)
Candidate key: (postal_code, house#, street)
Foreign key:
Constraints:
FDs:

```
CREATE TABLE location1(
        POSTAL_CODE              CHAR(20),
        HOUSE#                   INTEGER,
        STREET                   CHAR(20),

        PRIMARY KEY (POSTAL_CODE, HOUSE#, STREET)
)
```

T14
location2(postal_code: string, city: string)
Primary key: (postal_code)
Candidate key: (postal_code)
Foreign key:
Constraints:
FDs:
postal_code->city

```
CREATE TABLE location2(
```

```
        POSTAL_CODE                 CHAR(20),
        CITY                        CHAR(20),

        PRIMARY KEY (POSTAL_CODE)
)
```

T15
location3(postal_code: string, province: string)
Primary key: (postal_code)
Candidate key: (postal_code)
Foreign key:
Constraints:
FDs:
postal_code->province

```
CREATE TABLE location3(
        POSTAL_CODE                 CHAR(20),
        PROVINCE                    CHAR(20),

        PRIMARY KEY (POSTAL_CODE)
)
```

T16
user_account(user_account_id: integer, cooking_level: integer, postal_code: string, house#: integer, street: string)
Primary key: (user_account_id)
Candidate key: user_account_id
Foreign key: postal_code, house#, street
Constraints: house# cannot be NULL
postal_code cannot be NULL
street cannot be NULL

FDs:

```
CREATE TABLE user_account(
        USER_ACCOUNT_ID         INTEGER,
        COOKING_LEVEL           INTEGER,
        POSTAL_CODE             CHAR(20) NOT NULL,
        HOUSE#                  INTEGER NOT NULL,
        STREET                  CHAR(20) NOT NULL,


       PRIMARY KEY (USER_ACCOUNT_ID)
        FOREIGN KEY (POSTAL_CODE) REFERENCES location(POSTAL_CODE),
        FOREIGN KEY (HOUSE#) REFERENCES location(HOUSE#),
```

```
        FOREIGN KEY (STREET) REFERENCES location(STREET),
        FOREIGN KEY (USER_ACCOUNT_ID) REFERENCES account(ACCOUNT_ID)

)
```

T17
manager-account: (manager_account_id: integer)
Primary key: manager_account_id
Candidate key: manager_account_id
Foreign key: manager_account_id
Constraints:
FDs:

```
CREATE TABLE manager_account(
        MANAGER_ACCOUNT_ID              INTEGER



        PRIMARY KEY, MANAGER_ACCOUNT_ID
        FOREIGN KEY (MANAGER_ACCOUNT_ID) REFERENCES account(ACCOUNT_ID),
)
```

7. Instance

recipe

ID	Difficult(1-5)	CreateTime	User_ID	Method_ID	Region_ID
1	1	20210607	111069	1	14
2	1	20210523	111001	1	35
3	4	20210607	111078	4	25
4	5	20210821	1111002	5	14
5	3	20200524	11145	34	15

typeByMethod

ID	Method
1	fry
2	boil
4	steam
5	stew
34	grill

typeByRegion1

ID	local_dish
14	Sichuan cuisine
35	Vancouver cuisine
25	Contonese cuisine
14	Quebec cuisine
15	Osaka cuisine

typeByRegion2

local_dish	country
Sichuan cuisine	China
Vancouver cuisine	Canada
Contonese cuisine	China
Quebec cuisine	Canada
Osaka cuisine	Japen

photo_includes

photo_ID	recipe_ID
1	2
1	3
2	3
4	5
16	26

rating

recipe_ID	user_ID	score(1-5)
2	4	3
3	6	5
3	4	5
5	7	1
26	55	2

ingredient_name

ingredient_name
onion
tomato
potato
carrot
lettuce

contain

ingredient_name	recipe_ID
onion	2
tomato	3
potato	3
carrot	5
lettuce	26

account

account_ID	username	Email	password
4	Four	4@gmail.com	44
6	Six	6@gmail.com	66
8	Eight	8@gmail.com	8888
7	Seven	7@gmail.com	77
55	Fifty Five	55@gmail.com	5555

search

account_ID	ingredient-name	counter
4	onion	3
6	tomato	3
8	onion	1
7	carrot	22
55	lettuce	3

comment

id	createTime	recipe_ID	user_account_id
1	20210607	1	4
2	20210523	2	66
3	20210607	4	4
4	20210821	4	15
5	20200524	5	6

Favorite

user_account_id	recipe_id
4	1
66	2
7	4
15	4
6	5

Location1

postalCode	house#	street
v4sk7x	3333	11th
x5mk3l	2222	33th
n4ma1b	3333	10th
k3hb6s	6666	9th
n3ps9v	9999	4th

Location2

postalCode	City
v4sk7x	vancouver
x5mk3l	tronto
n4ma1b	BaoDing
k3hb6s	XiaMen
n3ps9v	Montreal

Location3

postalCode	province
v4sk7x	BC
x5mk3l	ON
n4ma1b	HeiBei
k3hb6s	FuJian
n3ps9v	QC

userAccount

user_account_id	cooking_level(1-5)	postalCode	house#	street
4	3	v4sk7x	3333	11th
66	5	x5mk3l	2222	33th
7	1	n4ma1b	3333	10th
15	3	k3hb6s	6666	9th
6	2	n3ps9v	9999	4th

managerAccount
manager_account_id
1
2
3
4
5

8. Queries

Insertion:

1. Add a recipe to the recipe table.
2. Add a method type to the typeByMethod table.
3. Add a local dish to the typeByRegion1 table.
4. Add a local dish to the typeByRegion2 table.
5. Add a photo to the photo table.
6. Add a rating written by a user for a recipe to the rating table.
7. Add an ingredient with its ingredient name to the ingredient table.
8. Add the recipe and the included ingredients to the contain table.
9. Add an account with Account_iD, username, Email, password to the account table.
10. Add a user ingredient search record to the search table.
11. Add a comment to the recipe by the user.
12. Add the recipe to the user's favorite list.
13. Add the user’s location information to the location1 table.
14. Add the user’s location information to the location2 table.
15. Add the user’s location information to the location3 table.
16. Add the new user account and its information to the existing accounts database.
17. Add(set) the manager account to the existing accounts database.

Deletion:

1. delete a recipe with id1 from the recipe table.
2. delete a method type with id1 from the typeByMethod table.
3. delete a local dish with id1 from the typeByRegion1 table.(?)
4. delete all chinese cuisine from the typeByRegion2 table.(?)
5. delete a photo with id1 from the photo table.
6. delete a rating written by a user with id 1 for a recipe with id 1 from the rating table.
7. delete an ingredient with its ingredient name from the ingredient table.
8. Delete a recipe and the included ingredients from the contain table..
9. Delete an account with Account_iD, username, Email, password from the account table.
10. Delete an user ingredient search record from the search table.
11. Delete the comment on the recipe that has been commented on by the user.
12. Delete the recipe from the user’s favorite list.
13. Delete user’s location information from the location1 table
14. Delete user’s location information from the location2 table
15. Delete user’s location information from the location3 table
16. Delete the user account from the existing accounts database.
17. Delete the manager account from the existing account database.